

```

• void LateUpdate () {
•     transform.position = new Vector3(Mathf.
Clamp(transform.position.x, -8f, 8f),
•     Mathf.Clamp(transform.position.y, -4.2f, 4.2f),
transform.position.z);
• }
• }

```

Note that the public class name should be the same as the file name of your script, or else it will not work. Let us understand the script by going through it line by line. Whatever is there after the `//` is used as a comment to explain what is happening to others who read the code later on. In large teams with complicated projects, these comments are essential in getting developers familiar with the code you have written. The `void Start()` section is usually used to define variables, but we do not need any here. We have changed the `Update ()` function to the `LateUpdate ()` function. What this does is changes the desired value after a keypress, instead of once every frame, which is done by the `Update ()` function. Next, we have the `transform.position` value. This corresponds to the `Position Attribute` in the `Transform Component` on the `GameObject`. Changing them in the script, through calculations, also changes their values in the associated `Components`. The new `Vector3` can hold positions and direction of movement in 3 dimensions. These three dimensions are `x,y` and `z` axes. Now `transform.position.x` and `transform.position.y` correspond to the `X` and `Y` values of the `Position Attribute`. There are three parameters in each `Mathf.Clamp` function. The first is the desired value, the second is the minimum possible value, and the third is the maximum possible value. To get these values, we simply moved the ship in the screen mode using the `transform` tool, and checked the values at the edges of the screen, then used the same values in the script. The `transform.position.z` value remains unchanged, as the ship cannot move along the `z` axis anyway.

For our space shooter, we will be allowing the ship to move in all four directions. Some may prefer to only allow the spaceship to move along the `Y` axis, and not move forward and backwards at all. There are many approaches to achieving this, and we will be showing you three. The first two can be done by merely changing the values of `Attributes` in the `Components` that we have already added to the `player-ship GameObject`, and the last one is through a change in the `bound script`. The first is through the `Movement Type attribute` in the `Move (Script) component`. The value can be changed to